PART 1 | CHALLENGE BRIEF

# Full Stack Engineering Hackathon

Collaborative Project Management Platform

> **AI tools are allowed.** You may use ChatGPT, GitHub Copilot, Cursor, Claude or similar tools. You are expected to understand and be able to explain everything you submit.

## 1. Objective

Build a multi-user collaborative project management platform where users with different roles can manage projects, tasks, team members, comments, and notifications. Relevant changes made by one connected user must be reflected to other users in real time, without requiring a manual page refresh.

## 2. User Roles

| Role | Responsibilities |
|---|---|
| **Admin** | Manage users; create/delete projects; assign project managers; view all projects and system activity. |
| **Project Manager** | Manage assigned projects; add/remove team members; create/update/delete tasks; assign tasks; change task status; view project activity. |
| **Team Member** | View projects they belong to; view assigned tasks; update their tasks; add comments; change task status. |
| **Viewer** | Read-only access to projects, tasks, and activity. |

## 3. Suggested Application Pages

| | | |
|---|---|---|
| `/login` | `/dashboard` | `/projects` |
| `/projects/:id` | `/projects/:id/tasks` | `/projects/:id/team` |
| `/projects/:id/activity` | `/tasks/:id` | `/users (Admin)` |
| `/notifications` | `/profile` | |

The exact UI and information architecture are intentionally left to you.

## 4. Core CRUD Requirements

- **Users:** create, read, update, deactivate.
- **Projects:** create, read, update, archive.
- **Team members:** add, remove, change project-level role.
- **Tasks:** create, read, update, delete, assign, change status.
- **Comments:** create, read, delete.

## 5. Real-Time Collaboration

Changes made by one connected user must be reflected to other relevant connected users without a page refresh. This applies to task creation/updates, status changes, assignments, comments, and relevant notifications.

The real-time approach is your choice (WebSockets, Socket.IO, SSE, a managed service, etc.). Choose what you consider appropriate and be ready to justify it.

## 6. Authentication & Authorization

Authentication is required, but **authorization is the key requirement**. Permissions must be enforced **on the server**, not only by hiding frontend buttons. For example, a Team Member attempting an unauthorized API operation directly must be rejected.

## 7. Concurrent Updates

Two users may open and edit the same task simultaneously. The system must prevent a stale update from silently overwriting a newer one.

> **Example:** Task version = 7. User A updates it and the server moves the version to 8. User B then submits an update based on version 7. The backend should detect the conflict and return an appropriate response such as **HTTP 409**, with the UI guiding the user to review the latest state.

## 8. Activity Log

Important actions must produce an activity record, for example: project created, member added, task created, task assigned, status changed, and comment added.

**Suggested fields:** id, user_id, project_id, action, entity_type, entity_id, metadata, created_at.

## 9. Notifications

Relevant users should receive notifications for events such as task assignment, status changes, mentions, project membership changes, and comments. Notifications must support unread/read state and should update in real time when appropriate.

## 10. Data Model

The database schema is not provided. Design it from the business requirements. A reasonable design may include: *users, roles, projects, project_members, tasks, comments, notifications,* and *activity_logs*.

## 11. API Design

Design your own API contract. Pay attention to naming, HTTP semantics, validation, authorization, error handling, pagination, filtering, and consistency.

## 12. AI Usage Policy

AI coding assistants are allowed and encouraged. You must understand and be able to explain all submitted code and architectural decisions, including which parts AI helped build and which engineering decisions you made yourself. You may be asked to modify, debug, or extend parts of your implementation.

PART 2 | GUIDELINES & PREREQUISITES

# Hackathon Guidelines & Prerequisites

Full Stack Engineering Hackathon - Collaborative Project Management Platform

> Read this document before the hackathon begins. Set up your laptop, verify the checklist in section 2, and be ready to start building on day one. This part complements the **Challenge Brief (Part 1)**.

## 1. Recommended Technology Stack

We primarily work with the following stack, so we recommend it. If you are strongest in something else, you may use it, but make sure you can fully explain and run it.

| Layer | Recommended | Notes |
|---|---|---|
| **Frontend** | Next.js (App Router preferred) | React-based UI, routing and pages. |
| **Language** | TypeScript (preferred) or JavaScript | TypeScript is encouraged for safer APIs and data models. |
| **Styling** | Tailwind CSS | Utility-first styling for the whole UI. |
| **Backend** | Express.js, NestJS (Node.js) or Python FastAPI | A separate API server that owns business logic, authorization and real-time connections. |
| **Database** | PostgreSQL + SQL (preferred) | Any database of your choice is accepted. Relational modeling suits this problem well. |
| **Data access** | Your choice | Raw SQL, Prisma, Drizzle, TypeORM, SQLAlchemy, etc. You must understand the queries and schema you ship. |

## 2. Laptop Prerequisites

Install and verify the following **before** the hackathon. Do not spend hackathon time on installations.

| Software | Version | Purpose | Verify with |
|---|---|---|---|
| **Node.js** | 22+ | Runtime for Next.js, Express and NestJS | `node -v` |
| **Python (if using FastAPI)** | 3.11+ | Runtime for the FastAPI backend | `python --version` |
| **Package manager** | npm (bundled), pnpm or yarn | Installing dependencies | `npm -v` |
| **Git** | Latest | Version control; commit regularly | `git --version` |
| **Code editor** | VS Code, Cursor or similar | Development | - |
| **PostgreSQL** | 14+ (15+ recommended) | Primary database (or use Docker / a hosted instance) | `psql --version` |

| Software | Version | Purpose | Verify with |
|---|---|---|---|
| **Database client** | pgAdmin, DBeaver, TablePlus or psql | Inspect tables and run queries | - |
| **API client** | Postman, Insomnia or Bruno | Test endpoints and authorization directly | - |
| **Browser** | Latest Chrome, Edge or Firefox | Testing; open two sessions to check real-time sync | - |
| **Docker (optional)** | Latest Docker Desktop | Run PostgreSQL without a local install | `docker -v` |
| **GitHub account** | - | Hosting your repository | - |

You will also need a stable internet connection, a charged laptop with a charger, and a mouse if you prefer one. Free disk space of at least 5 GB is recommended.

## 3. Pre-Hackathon Readiness Checklist

- Run `node -v` (22 or higher), `git --version` and `psql --version` successfully.
- Create a blank project with `npx create-next-app@latest` and confirm it runs locally.
- Start a hello-world API with your chosen backend (Express, NestJS or FastAPI) and call it from the Next.js app.
- Create an empty PostgreSQL database and confirm you can connect to it from your DB client.
- Confirm you can run two browser sessions (for example, a normal window and an incognito window) logged in as different users.
- Have your AI tools signed in and ready (optional).
- Make sure your Git is configured: `git config user.name` and `user.email`.

## 4. Libraries & Tools You May Use

You are free to use any third-party libraries, component kits and animation libraries. Using them is encouraged where it helps you build a better product faster. Examples:

| Category | Examples |
|---|---|
| **UI components** | shadcn/ui, Radix UI, Headless UI, daisyUI, Ark UI |
| **Animated components** | Magic UI, Aceternity UI, Framer Motion (Motion), React Bits |
| **Icons** | lucide-react, react-icons, Heroicons |
| **Forms & validation** | React Hook Form, Zod, Valibot |
| **Data fetching & state** | TanStack Query, SWR, Zustand |
| **Tables & charts** | TanStack Table, Recharts, Chart.js |
| **Auth** | Passport.js, NestJS guards, FastAPI security utilities, Auth.js, or your own JWT / session implementation |

| Category | Examples |
|---|---|
| **Backend frameworks** | Express.js, NestJS, FastAPI (with Pydantic for validation) |
| **ORM / query builders** | Prisma, Drizzle, TypeORM, Kysely, node-postgres (pg), SQLAlchemy / SQLModel with Alembic |
| **Real-time** | WebSockets (ws, Socket.IO, FastAPI WebSockets), Server-Sent Events, or a managed service - your choice |

> **Frontend and backend:** your Next.js app and your API server will run as separate processes. Plan early how they connect, including the API base URL, CORS, authentication tokens or cookies, and where your real-time connection is established.

## 5. Development Guidelines

- **Use Git properly.** Commit early and often with clear messages. Your commit history shows how you worked.
- **Never commit secrets.** Keep credentials in `.env`, add it to `.gitignore`, and provide a `.env.example`.
- **Provide seed data.** Include a seed script that creates sample users for each role (Admin, Project Manager, Team Member, Viewer) along with a few projects and tasks, so the app can be demonstrated quickly.
- **Use database migrations.** The schema should be reproducible from scratch with documented commands.
- **Enforce rules on the server.** Validate inputs and check permissions in the backend, not only in the UI.
- **Handle errors gracefully.** Return meaningful status codes and show clear messages in the UI.
- **Keep the code organised.** Use a sensible folder structure and consistent naming.
- **Test what matters.** Add tests for critical logic such as permissions and conflict detection, if time allows.

## 6. What to Submit

- A Git repository (GitHub or similar) containing the complete source code for the frontend and backend.
- A `README.md` with setup steps, environment variables, how to run migrations and the seed script, how to start both the frontend and the backend, and the sample login credentials.
- A short note on your architecture: the real-time approach you chose, your data model, and how authorization and concurrent updates are handled.
- A working application that can be run locally by following your README.